Practical 4: Minimum spanning Tree

Prim 's algorithm

NAME – ANANYA JADHAV

PRN – 202401040097
ROLL NO – 62
DIVISION – A
BATCH – A4

CODE :

// C++ implementation of Prim's Algorithm to find the Minimum Spanning tree for a weighted, connected and undirected graph.

```
#include <iostream>
#include <climits>
#define n 6
using namespace std;

// Printing the MST
void printMST(int a[n], int b[n], int weight[n])
{
   int Minweight = 0; // Weight of Minimum spanning tree
```

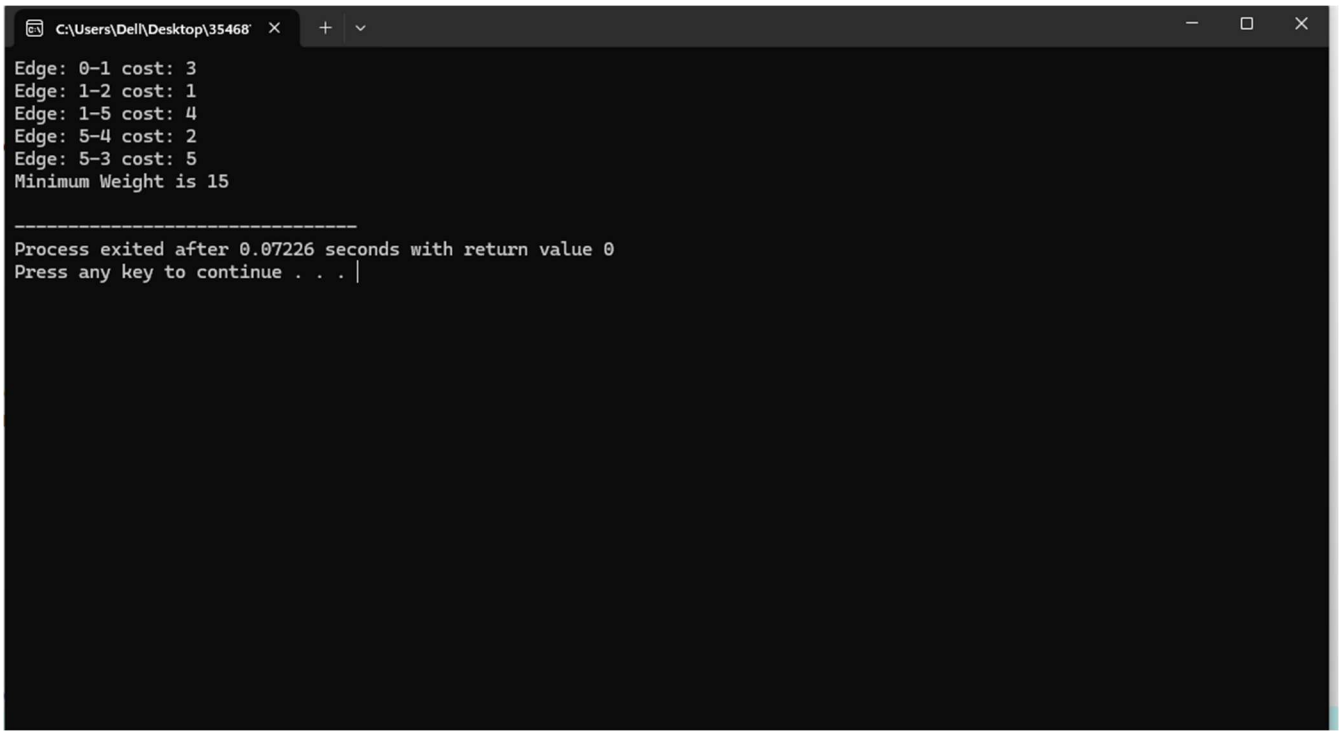
```
  for (int i = 0; i < n - 1; i++)
  {
    cout << "Edge: " << a[i] << "-" << b[i] << " "
      << "cost: " << weight[i] << endl;
    Minweight += weight[i];
  }
  cout << "Minimum Weight is " << Minweight << endl; // Printing the weight of MINIMUM SPANNING TREE
}

void prim(int cost[n][n]) // Function performing prim's algorithm
{
  int u, v, k = 0, counti = 0;
  int visited[n] = { 0 }; // Array containing all the nodes, Initialize with zero as they are not included in MST
  int a[n]; // Array containing the first nodes of all the edges present in MST
  int b[n]; // Array containing the first nodes of all the edges present in MST
  int weight[n]; // Array containing the weights of the edges present in MST
  int minimum;

  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      if (cost[i][j] == 0) //  If i and j nodes are not adjacent
        cost[i][j] = INT_MAX; // Then, initialize them as INFINITE

  visited[0] = 1; // Including the first vertex in MST
```

```
  while(counti < n-1)
  {
    minimum = INT_MAX; // Initializing minimum as INFINITE
    for (int i = 0; i < n; i++)
    {
      for (int j = 0; j < n; j++)
      {
        if (visited[i] != 0 && cost[i][j] < minimum) // If the first node is not in MST yet, traverse through all the edges connected through it
        {
          minimum = cost[i][j]; // To find the minimum cost edge
          u = i; // First node of determined edge
          v = j; // Second node of determined edge
        }
      }
    }

    if (visited[u] == 0 || visited[v] == 0) // If the node is not yet included in MST, then include it in MST
    {
      a[k] = u; // Store first node in array
      b[k] = v; // Store second node in array
      weight[k] = cost[u][v]; // Store the determined edge's weight in array
      counti++;
      k++;
      visited[v] = 1; // Vertex included in MST
    }
```

```
    cost[u][v] = cost[v][u] = INT_MAX; // Edges getting included in MST will be given the weight of INFINITE
  }

  printMST(a, b, weight); // Printing the MST
}

// Driver program to test above function
int main()
{

  int cost[n][n] = {
    { 0, 3, 0, 0, 6, 5 },
    { 3, 0, 1, 0, 0, 4 },
    { 0, 1, 0, 6, 0, 4 },
    { 0, 0, 6, 0, 8, 5 },
    { 6, 0, 0, 8, 0, 2 },
    { 5, 4, 4, 5, 2, 0 }
  };

  prim(cost); // Calling prim function
```

```
    return 0;
}


/*
Output :
Edge: 0-1 cost: 3
Edge: 1-2 cost: 1
Edge: 1-5 cost: 4
Edge: 5-4 cost: 2
Edge: 5-3 cost: 5
Minimum Weight is 15
*/
```

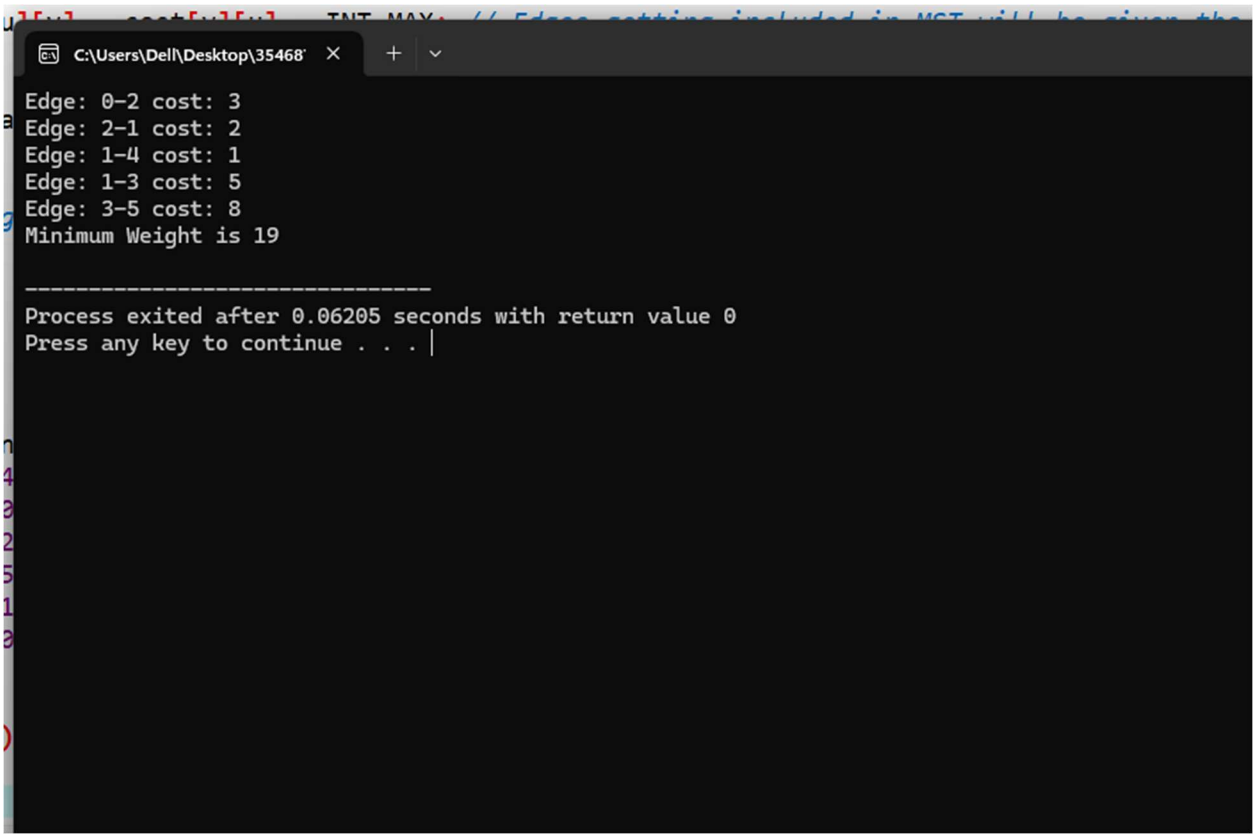
OUTPUT :



EXAMPLE 1 :

```
int cost[n][n] = {
      { 0, 4, 3, 0, 0, 0 },
      { 4, 0, 2, 5, 1, 0 },
      { 3, 2, 0, 0, 6, 0 },
      { 0, 5, 0, 0, 7, 8 },
      { 0, 1, 6, 7, 0, 9 },
      { 0, 0, 0, 8, 9, 0 }
   };
```

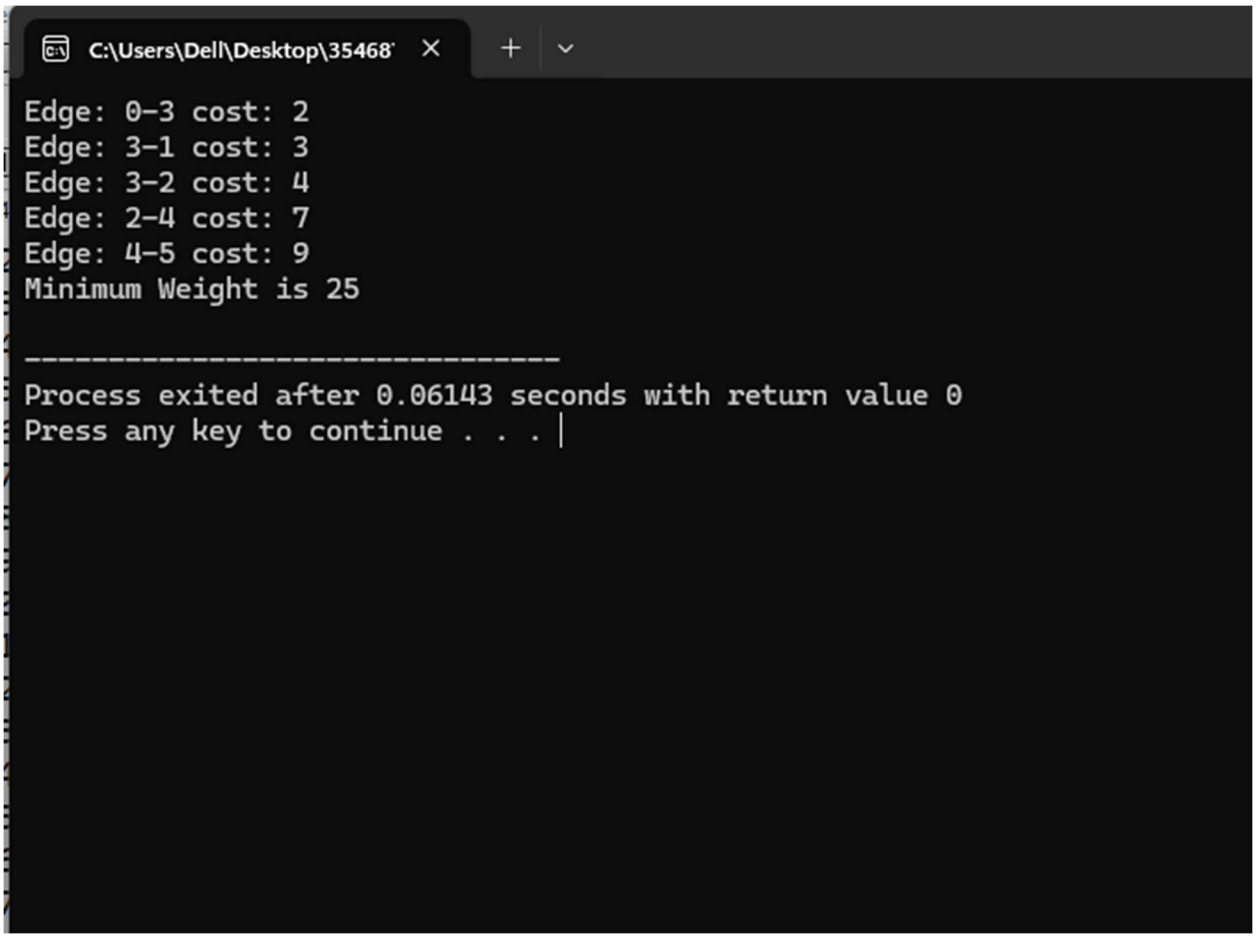
OUTPUT :



EXAMPLE 2 ::

```
int cost[n][n] = {
    { 0, 6, 5, 2, 0, 0 },
    { 6, 0, 0, 3, 0, 0 },
    { 5, 0, 0, 4, 7, 0 },
    { 2, 3, 4, 0, 8, 0 },
    { 0, 0, 7, 8, 0, 9 },
    { 0, 0, 0, 0, 9, 0 }
                        };
```

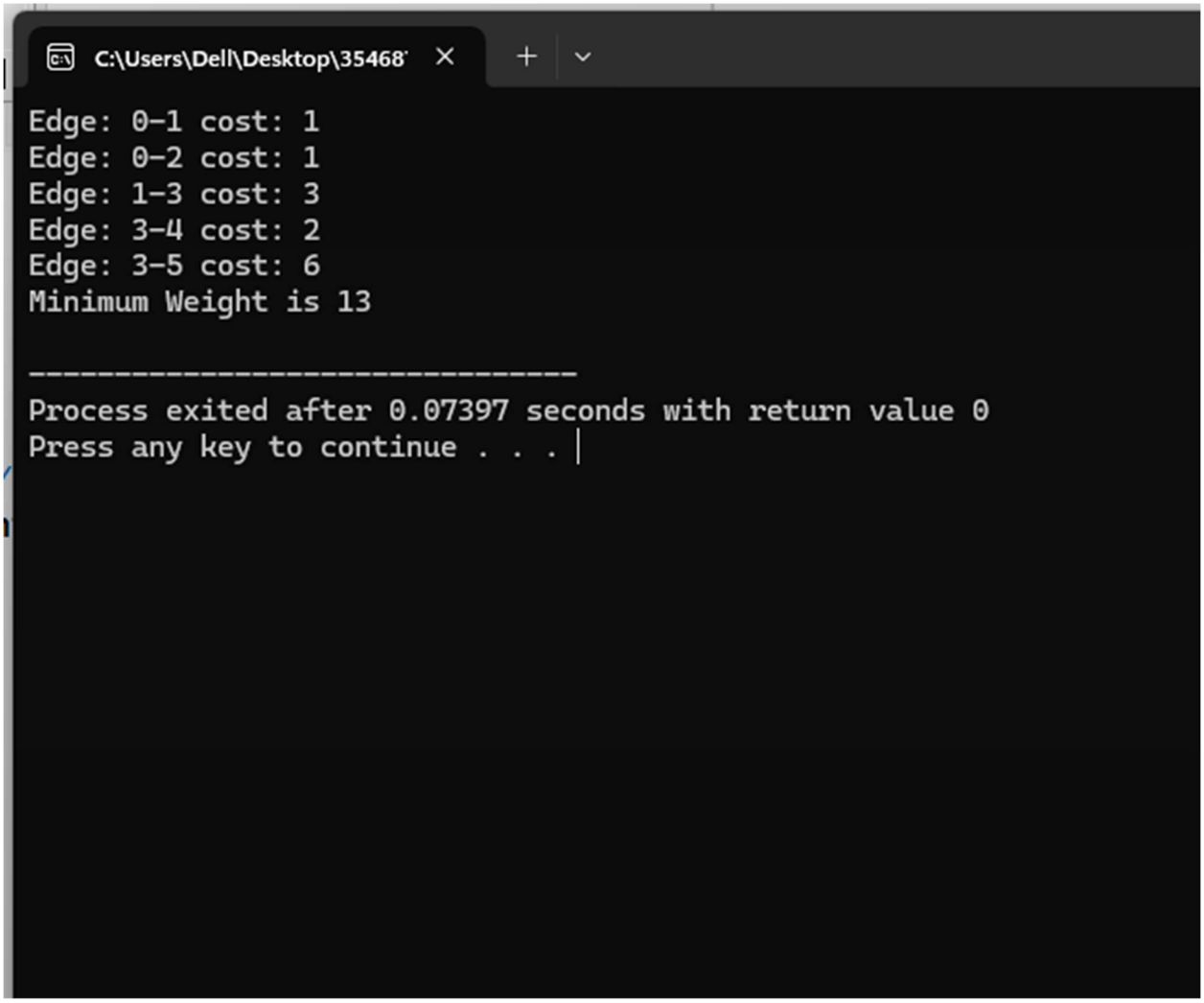
OUTPUT :



EXAMPLE 3 ::

```
int cost[n][n] = {
    { 0, 1, 1, 0, 0, 0 },
    { 1, 0, 2, 3, 4, 0 },
    { 1, 2, 0, 0, 5, 0 },
    { 0, 3, 0, 0, 2, 6 },
    { 0, 4, 5, 2, 0, 0 },
    { 0, 0, 0, 6, 0, 0 }
  };
```

OUTPUT :


C:\Users\Dell\Desktop\35468
Edge: 0-1 cost: 1
Edge: 0-2 cost: 1
Edge: 1-3 cost: 3
Edge: 3-4 cost: 2
Edge: 3-5 cost: 6
Minimum Weight is 13

Process exited after 0.07397 seconds with return value 0
Press any key to continue . . .